Assignment 5 - Concurrent Data Structures (2)

Learning Goals

This assignment extends the study of concurrent data structures from lock-based designs to advanced lock-free and non-blocking algorithms. In the previous assignment, you examined coarse-grained and fine-grained locking. In this assignment, you will focus on the design, implementation, correctness, and performance behavior of modern lock-free data structures under different contention regimes and machine-level effects.

The main goal is not only to implement a lock-free data structure that is functionally correct, but also to understand when and why lock-free algorithms perform well or poorly. In particular, you will study the interaction between algorithmic design and hardware effects such as cache coherence, false sharing, compare-and-swap retry behavior, and NUMA locality.

Generative AI Policy

This assignment allows the using of Generative AI tools on specific sub-tasks. Please follow the described instructions and use the tools only when necessary. If you used any Generative AI tools in this assignment, include the conversation (exported link) and discuss what you learned.

Part A: Lock-Free Stack

A0. Lock-Based stack as baseline

Task: Implement a simple stack protected by a single mutex. The data structure supports `pop(value)` and `push(value)` operations. This serves as a baseline to be compared with lock-free implementations.

A1. Treiber stack

The Treiber stack algorithm is a scalable lock-free stack utilizing the fine-grained concurrency primitive compare-and-swap. It was published by R. Kent Treiber in 1986.

The basic principle for the algorithm is to only add something new to the stack when the item to add is verified to be the only thing that has been added since the operation began. This is done by using compare-and-swap (CAS). Pushing an item to the stack is done by first taking the top of the stack (old head) and placing it after the new item to create a new head. The old head is then compared to the current head. If the two match then the old head can be swapped to the new one, if not then it means another thread has added an item to the stack, in which case another attempt is necessary. When popping an item from the stack, before returning the item it must be verified that another thread has not added a new item since the operation began.

Task: Implement a lock-free stack using a linked list and atomic CAS on the head pointer.

Hint: You can refer to Chat-GPT/Gemini/Claude on how to implement a Treiber stack. Please cite the conversation and explain in the report how those AI tools are used.

A2. Elimination-Backoff stack

Elimination-Backoff stack is a highly scalable, lock-free concurrent data structure that reduces contention by allowing opposing operations (pushes and pops) to cancel each other out in an "elimination array" rather than accessing a central stack simultaneously.
Task: Implement an elimination-backoff stack in which complementary `push` and `pop` operations may exchange values through elimination slots rather than always contending on the central stack head. You should design an elimination array and a simple policy for choosing slots and retry limits. The goal is to reduce contention on the stack head under high thread counts.
Hint: You can refer to Chat-GPT/Gemini/Claude on how to implement a Elimination-Backoff stack. Please cite the conversation and explain in the report how those AI tools are used.

Part B: Lock-Free Queue

B0. Lock-Based queue as baseline

Implement a queue protected by a single mutex for comparison. The data structure supports `enqueue(value)` and `dequeue(value)`, serving as the baseline for lock-free implementation.

B1. Michael–Scott queue

The Michael-Scott queue is a highly efficient, concurrent, lock-free, FIFO data structure designed by Maged M. Michael and Michael L. Scott. It uses atomic primitives, specifically Compare-And-Swap (CAS), to allow multiple threads to safely enqueue and dequeue items simultaneously without locks. It is widely used in high-performance computing, notably within the Java Concurrency Package.
Task: Implement Michael–Scott lock-free queue using atomic pointer updates on head and tail.
Hint: You can refer to Chat-GPT/Gemini/Claude on how to implement a Michael–Scott queue. Please cite the conversation and explain in the report how those AI tools are used.

Required Interfaces

Stack interface

```
typedef struct stack stack_t;

void stack_init(stack_t *s);
void stack_push(stack_t *s, int value);
int stack_pop(stack_t *s, int *value); // returns 1 on success, 0 if empty
void stack_destroy(stack_t *s);
```

Queue interface

```
typedef struct queue queue_t;

void queue_init(queue_t *q);
```

```
void queue_enqueue(queue_t *q, int value);
int queue_dequeue(queue_t *q, int *value); // returns 1 on success, 0 if empty
void queue_destroy(queue_t *q);
```

Experimental Requirements

You need to evaluate your implementations by running and measure the performance under various scenarios. You can develop a unified benchmark suite for easy evaluation. Such a benchmark might look like as follows.

```
./bench_stack --threads 8 --ops 1000000 --impl treiber --workload balanced
./bench_stack --threads 8 --ops 1000000 --impl elimination --workload balanced
./bench_queue --threads 8 --ops 1000000 --impl ms --workload mp_mc
./bench_queue --threads 8 --ops 1000000 --impl advanced --workload mp_mc
```

The arguments represent number of threads (`threads`), number of operations (`ops`), implementation variants (`impl`), workload type (`workload`).

C.1 Thread scaling Run experiments when scaling the number of threads as 1, 2, 4, 8, 16, 32.

C.2 Workload types For Stack workloads, use at least the following three kind of workloads: 1) *Push-heavy*: 80% push, 20% pop; 2) *Balanced*: 50% push, 50% pop; 3) *Pop-heavy*: 20% push, 80% pop. For Queue workloads, use at least the following two kinds of workloads: 1) *MPMC*: all threads both enqueue and dequeue 2) *Producer-consumer split*: half producer threads, half consumer threads.

You need to collect the end-to-end run time of the benchmark under each configuration (thread counts, workload types) as part of the deliverables. All measurements need to be collected by running jobs on Centaurus.

Deliverables

Source code Include 1) algorithm implementation source code, 2) benchmarking script and 3) the build script.

Benchmark results Include the time measurement results (presented in tables or figures). In particular, you should highlight your findings on 1) throughput versus thread count 2) runtime versus thread count.

Summary and discussion Interpret the results and connect them to the algorithmic design and machine behavior.

Report

A PDF document containing all deliverables. Hint: organize your report with the performance measurements first, then the summary and discussion, followed by source code.

Report format

The assignment report follows a modified ACM conference proceedings template. You may use the templates available in LaTeXor Overleaf, which can be accessed here. Assignment submissions that does not follow the designated templates will be penalized with up to 20% points deduction.